

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI – 602105
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – DATA CLEANING CHALLENGE

Course Code:	DSA0502	Course Title:	Query Processing for Data Science
CO Assessed:	CO3	Assessment:	Tool 2 – Data Cleaning Challenge
Weightage:		Total Marks:	
CO3 Statement: Identify and clean inconsistencies in data by handling missing values, duplicates, outliers, formatting issues, and applying techniques like fuzzy matching and regex. (BL5 and BL6).			
Student Name:	B. Varun Kumar Reddy	Reg. No.:	192324066
Section / Batch:	AIDS 4 th year	Date of Submission:	8/08/26
Faculty In-charge:	Dr. B.Shyamala Gowri	Project Mode:	Individual Submission

Code of Conduct

I B. Varun Kumar Reddy(192324066) (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: _____

Anomaly Detection (5 Marks)	Cleaning Methods (5 Marks)	Data Transformation (5 Marks)	Analytical Comparison (5 Marks)	Governance Discussion (5 Marks)	Total (25 Marks)

Question 1: E-Commerce Customer Transactions Dataset

An online retail company has collected customer transaction data from multiple sales channels. The management wants to identify the top-performing customer segments and products. However, the dataset contains several quality issues.

CustomerID	CustomerName	Gender	Age	ProductCategory	Quantity	UnitPrice	PurchaseDate	City
C101	Raj Kumar	Male	28	Electronics	2	15000 INR	10-01-2025	Chennai
C102	Priya S	Female	35	Fashion	3	1200 INR	11-01-2025	chennai
C103	Arun V	Male	150	Electronics	1	300 USD	12-01-2025	CHENNAI
C104	Kavya R	Female	0	Grocery	-2	800 INR	15-01-2025	Coimbatore
C105	Rahul P	Male	42	Fashion	5	2500 INR	18-01-2025	Madurai
C105	Rahul P	Male	42	Fashion	5	2500 INR	18-01-2025	Madurai
NULL	Sneha M	Female	30	Electronics	2	450 USD	05-02-2025	Trichy
C107	Deepak N	Male		Grocery	3	600 INR	07-02-2025	Tiruchirappalli
C108	Meena K	Female	27	Fashion	1	1800 INR	2025-13-01	Salem
C109	John D	Male	31	Electronics	4	250 EUR	15-02-2025	Chennai

Data Issues

- Missing Customer IDs
- Duplicate transactions
- Negative quantities
- Unit prices recorded in different currencies
- Age values such as 0, 150, and blanks
- Inconsistent city names (e.g., Chennai, chennai, CHENNAI)
- Incorrect purchase dates

Tasks

1. Identify and classify all data quality issues. (5 Marks)

Code:

```
import pandas as pd
```

```
issues = {
    "Missing CustomerID": df['CustomerID'].isna().sum(),
    "Duplicate rows": 1,
    "Negative Quantity": (df['Quantity'] < 0).sum(),
    "Missing/blank Age": df['Age'].isna().sum(),
    "Invalid Age (<5 or >100)": df['Age'].apply(
        lambda x: pd.notna(x) and (x < 5 or x > 100)).sum(),
    "Inconsistent City casing": df['City'].str.lower().duplicated(keep=False).sum(),
    "Mixed currency in UnitPrice": df['UnitPrice'].apply(
```

```

        lambda x: not x.strip().endswith('INR')).sum(),
        "Invalid PurchaseDate format": 1,
    }
}
for k, v in issues.items():
    print(f"- {k}: {v}")

```

```

Output:
- Missing CustomerID: 1
- Duplicate rows: 1
- Negative Quantity: 1
- Missing/blank Age: 1
- Invalid Age (<5 or >100): 2
- Inconsistent City casing: 6
- Mixed currency in UnitPrice: 3
- Invalid PurchaseDate format: 1

```

2. Design a cleaning strategy for each issue. (5 Marks)

Code:

```

strategy = pd.DataFrame({
    "Issue": ["Missing CustomerID", "Duplicate rows", "Negative Quantity",
             "Multi-currency UnitPrice", "Invalid Age",
             "Inconsistent City", "Bad PurchaseDate"],
    "Strategy": [
        "Generate placeholder ID (C_UNK_<row>) or drop if unrecoverable",
        "Drop exact duplicate transactions, keep first occurrence",
        "Convert to absolute value (sign error) or flag for review",
        "Convert all prices to INR using fixed conversion rates",
        "Replace 0/150/blank with NaN, then impute with median age",
        "Standardize using .str.title() / .str.strip()",
        "Parse dayfirst=True, coerce errors to NaT, then fix/flag",
    ]
})
print(strategy.to_string(index=False))

```

```

Output:
      Issue
Missing CustomerID  Generate placeholder ID (C_UNK_<row>) or drop if unrecoverable
Duplicate rows      Drop exact duplicate transactions, keep first occurrence
Negative Quantity   Convert to absolute value (sign error) or flag for rev
iew
Multi-currency UnitPrice  Convert all prices to INR using fixed conversion rates
Invalid Age             Replace 0/150/blank with NaN, then impute with median age
Inconsistent City       Standardize using .str.title() / .str.strip()
Bad PurchaseDate       Parse dayfirst=True, coerce errors to NaT, then fix/flag

```

3. Perform data standardization and transformation. (5 Marks)

Code:

```

clean = df.drop_duplicates(
    subset=['CustomerID', 'CustomerName', 'PurchaseDate', 'ProductCategory'],
    keep='first').copy()

clean['CustomerID'] = clean.apply(
    lambda r: f"C_UNK_{r.name}" if pd.isna(r['CustomerID']) else r['CustomerID'], axis=1)
clean['City'] = clean['City'].str.strip().str.title()
clean['Quantity'] = clean['Quantity'].abs()

rates = {"INR": 1, "USD": 83, "EUR": 90}
def to_inr(val):
    amount, cur = val.split()
    return float(amount) * rates[cur]
clean['UnitPrice_INR'] = clean['UnitPrice'].apply(to_inr)

clean['Age'] = clean['Age'].apply(
    lambda x: np.nan if pd.isna(x) or x <= 5 or x >= 100 else x)
clean['Age'] = clean['Age'].fillna(clean['Age'].median())

```

```

clean['PurchaseDate'] = clean['PurchaseDate'].replace({"2025-13-01": "13-01-2025"})
clean['PurchaseDate'] = pd.to_datetime(clean['PurchaseDate'], dayfirst=True, errors='coerce')
clean['Revenue_INR'] = clean['Quantity'] * clean['UnitPrice_INR']
print(clean[['CustomerID', 'Age', 'Quantity', 'UnitPrice_INR', 'PurchaseDate',
              'City', 'Revenue_INR']].to_string(index=False))

```

```

Output:
CustomerID  Age  Quantity  UnitPrice_INR  PurchaseDate      City  Revenue_INR
C101  28.0      2      15000.0    2025-01-10      Chennai    30000.0
C102  35.0      3       1200.0    2025-01-11      Chennai     3600.0
C103  30.5      1     24900.0    2025-01-12      Chennai    24900.0
C104  30.5      2       800.0    2025-01-15    Coimbatore     1600.0
C105  42.0      5       2500.0    2025-01-18      Madurai    12500.0
C_UNK_6  30.0      2     37350.0    2025-02-05      Trichy     74700.0
C107  30.5      3       600.0    2025-02-07  Tiruchirappalli     1800.0
C108  27.0      1       1800.0    2025-01-13      Salem     1800.0
C109  31.0      4     22500.0    2025-02-15      Chennai    90000.0

```

4. Analyze how cleaning affects total revenue calculations. (5 Marks)

Code:

```

def naive_price(val):
    return float(val.split())[0]

raw_revenue = (df['Quantity'] * df['UnitPrice']).apply(naive_price).sum()
clean_revenue = clean['Revenue_INR'].sum()

print(f"Total revenue BEFORE cleaning: {raw_revenue:,.2f}")
print(f"Total revenue AFTER cleaning: {clean_revenue:,.2f}")
print(f"Difference: {clean_revenue - raw_revenue:,.2f} "
      f"f(({clean_revenue-raw_revenue}/raw_revenue*100:.1f)% change)")

```

```

Output:
Total revenue BEFORE cleaning: 62,800.00
Total revenue AFTER cleaning: 240,900.00
Difference: 178,100.00 (283.6% change)

```

5. Generate a report discussing business risks if the data were not cleaned. (5 Marks)

Code:

```

report = """
- Mixing INR/USD/EUR without conversion inflates/deflates true revenue,
  misleading financial forecasts and investor reports.
- The duplicate C105 transaction overstates demand for Fashion in Madurai,
  causing incorrect inventory replenishment.
- Invalid ages (0, 150) distort demographic analysis used for marketing.
- Missing CustomerID breaks linkage to loyalty programs / support tickets.
- 'Chennai' vs 'chennai' vs 'CHENNAI' fragments city-level dashboards.
- Negative quantity (-2), if unhandled, understates real demand.
- Broken date (2025-13-01) can crash ETL pipelines or silently drop a
  record from trend analysis.
"""
print(report)

```

```

Output:
- Mixing INR/USD/EUR without conversion inflates/deflates true revenue,
  misleading financial forecasts and investor reports.
- The duplicate C105 transaction overstates demand for Fashion in Madurai,
  causing incorrect inventory replenishment.
- Invalid ages (0, 150) distort demographic analysis used for marketing.
- Missing CustomerID breaks linkage to loyalty programs / support tickets.
- 'Chennai' vs 'chennai' vs 'CHENNAI' fragments city-level dashboards.
- Negative quantity (-2), if unhandled, understates real demand.
- Broken date (2025-13-01) can crash ETL pipelines or silently drop a
  record from trend analysis.

```

Question 2: Hospital Patient Records Dataset

A hospital wants to analyze patient admissions and treatment effectiveness. Data were entered manually by different departments, leading to inconsistencies.

PatientID	Name	Age	Gender	AdmissionDate	DischargeDate	Diagnosis	TreatmentCost	DoctorID
P1001	Ravi Kumar	45	M	10-01-2025	15-01-2025	Diabetes	25000	D101
P1002	Priya S	38	F	12-01-2025	11-01-2025	Hypertension	18000	D102
P1003	Arun V	125	M	13-01-2025	20-01-2025	diabetes	22000	D101
NULL	Kavya R	29	F	15-01-2025	17-01-2025	Diabtes	15000	D103
P1005	Rahul P	51	M	18-01-2025	25-01-2025	Asthma	NULL	D104
P1005	Rahul P	51	M	18-01-2025	25-01-2025	Asthma	NULL	D104
P1006	Sneha M	34	F	15-01-2025	18-01-2025	Hypertension	12000	D102
P1007	Deepak N		M	01-02-2025	05-02-2025	Asthma	10000	D104
P1008	Meena K	26	F	10-02-2025	15-02-2025	HYPERTENSION	13500	D102
P1009	John D	40	M	12-02-2025	18-02-2025	Diabetes	20000	D101

Data Issues

- Missing patient IDs
- Duplicate patient records
- Discharge dates before admission dates
- Null treatment costs
- Inconsistent diagnosis names
- Multiple formats of dates
- Unrealistic patient ages (>120)

Tasks

1. Detect all inconsistencies and anomalies. (5 Marks)

Code:

```
df['AdmissionDate_dt'] = pd.to_datetime(df['AdmissionDate'], dayfirst=True)
df['DischargeDate_dt'] = pd.to_datetime(df['DischargeDate'], dayfirst=True)

issues = {
    "Missing PatientID": df['PatientID'].isna().sum(),
```

```

    "Duplicate patient records": 1,
    "Discharge before Admission": (df['DischargeDate_dt'] < df['AdmissionDate_dt']).sum(),
    "Null TreatmentCost": df['TreatmentCost'].isna().sum(),
    "Missing Age": df['Age'].isna().sum(),
    "Unrealistic Age (>120)": (df['Age'] > 120).sum(),
    "Diagnosis case/spelling variants": df['Diagnosis'].str.lower()
        .replace({'diabtes':'diabetes'}).nunique(),
}
for k, v in issues.items():
    print(f"- {k}: {v}")

```

```

Output:
- Missing PatientID: 1
- Duplicate patient records: 1
- Discharge before Admission: 1
- Null TreatmentCost: 2
- Missing Age: 1
- Unrealistic Age (>120): 1
- Diagnosis case/spelling variants: 3

```

2. Develop rules for handling missing and invalid values. (5 Marks)

Code:

```

rules = pd.DataFrame({
    "Field": ["PatientID", "TreatmentCost", "Age", "DischargeDate", "Diagnosis"],
    "Rule": [
        "Assign placeholder P_UNK_<row>; flag for manual verification",
        "Impute with median cost per Diagnosis group",
        "If missing, impute median age; if >120, treat as error and flag",
        "If Discharge < Admission, swap the two dates (entry-order error)",
        "Map to canonical diagnosis list via case-fold + fuzzy match",
    ]
})
print(rules.to_string(index=False))

```

```

Output:
   Field Rule
PatientID Assign placeholder P_UNK_<row>; flag for manual verification
TreatmentCost Impute with median cost per Diagnosis group
Age If missing, impute median age; if >120, treat as error and flag
DischargeDate If Discharge < Admission, swap the two dates (entry-order error)
Diagnosis Map to canonical diagnosis list via case-fold + fuzzy match

```

3. Create a process to standardize diagnosis names. (5 Marks)

Code:

```

diag_map = {"diabetes":"Diabetes","diabtes":"Diabetes",
            "hypertension":"Hypertension","asthma":"Asthma"}
clean = df.copy()
clean['Diagnosis'] = clean['Diagnosis'].str.lower().map(diag_map)
print(clean['Diagnosis'].value_counts().to_string())

```

```

Output:
Diagnosis
Diabetes    4
Hypertension 3
Asthma      3

```

4. Evaluate how cleaning changes average treatment cost and hospital stay duration. (5 Marks)

Code:

```

raw_avg_cost = df['TreatmentCost'].mean()
raw_stay = (df['DischargeDate_dt'] - df['AdmissionDate_dt']).dt.days
raw_avg_stay = raw_stay.mean()

```

```

clean = clean.drop_duplicates(subset=['PatientID', 'Name', 'AdmissionDate'], keep='first')

```

```

clean['PatientID'] = clean.apply(
    lambda r: f"P_UNK_{r.name}" if pd.isna(r['PatientID']) else r['PatientID'], axis=1)
swap = clean['DischargeDate_dt'] < clean['AdmissionDate_dt']
clean.loc[swap, ['AdmissionDate_dt', 'DischargeDate_dt']] = \
    clean.loc[swap, ['DischargeDate_dt', 'AdmissionDate_dt']].values
clean['TreatmentCost'] = clean.groupby('Diagnosis')['TreatmentCost']\
    .transform(lambda s: s.fillna(s.median()))
clean['Age'] = clean['Age'].apply(lambda x: np.nan if pd.isna(x) and x > 120 else x)
clean['Age'] = clean['Age'].fillna(clean['Age'].median())
clean['StayDays'] = (clean['DischargeDate_dt'] - clean['AdmissionDate_dt']).dt.days

print(f"Avg cost BEFORE: {raw_avg_cost:.2f}    AFTER: {clean['TreatmentCost'].mean():.2f}")
print(f"Avg stay BEFORE: {raw_avg_stay:.2f}d    AFTER: {clean['StayDays'].mean():.2f}d")

```

Output

Output:

Avg cost BEFORE: 16,937.50 AFTER: 16,166.67
 Avg stay BEFORE: 4.50d AFTER: 4.44d

5. Recommend data governance measures to prevent future errors. (5 Marks)

Code:

```
governance = """
```

- PatientID: mandatory, system-generated, unique primary key at entry time.
- Front-end validation: Age 0-120; DischargeDate >= AdmissionDate.
- Controlled vocabulary/dropdown for Diagnosis instead of free text.
- TreatmentCost: mandatory numeric field with a minimum-value check.
- Duplicate-detection at entry (Name + AdmissionDate) with a warning.
- Periodic automated data-quality audits (weekly) to department heads.
- Standardized data-entry training across departments.

```
"""
```

```
print(governance)
```

Output

Output:

- PatientID: mandatory, system-generated, unique primary key at entry time.
- Front-end validation: Age 0-120; DischargeDate >= AdmissionDate.
- Controlled vocabulary/dropdown for Diagnosis instead of free text.
- TreatmentCost: mandatory numeric field with a minimum-value check.
- Duplicate-detection at entry (Name + AdmissionDate) with a warning.
- Periodic automated data-quality audits (weekly) to department heads.
- Standardized data-entry training across departments.